Mastering Connect 4 via Deep Reinforcement Learning and Monte Carlo Tree Search

Final Project Report

Xinyi Yang (xinyi_yang@ucsb.edu)

University of California, Santa Barbara

April 3, 2026

Abstract

This report details the development of a Deep Reinforcement Learning (DRL) agent designed to master Connect 4, a classic game exhibiting a state-space complexity of approximately 4.5×10^{12}. Inspired by the AlphaZero architecture, our agent integrates Monte Carlo Tree Search (MCTS) with a Residual Convolutional Neural Network (CNN) to iteratively approximate optimal strategies via self-play, entirely devoid of human-derived heuristics. By utilizing the OpenSpiel framework for rigorous simulation, the agent was evaluated against Random, Greedy, and Alpha-Beta Minimax baselines (depth 4 and depth 6), ultimately achieving win rates of **95%**, **87%**, and **63%** respectively. This report documents the comprehensive training pipeline, network architecture, MCTS implementation, loss convergence, and empirical Elo rating progression.

Contents

1. Introduction

Connect 4 is a canonical two-player connection board game played on a 6×7 grid. Although it is a *strongly solved* game—mathematically proven by Allis [1] to be a first-player win under optimal play—its state-space complexity of 4.5×10^{12} reachable positions renders exhaustive search algorithms computationally prohibitive. Furthermore, the game tree complexity is approximately 10^{21}, strictly ruling out complete minimax enumeration in any practical, unconstrained setting.

This project implements an intelligent agent that learns an optimal Connect 4 strategy entirely through self-play, directly inspired by the AlphaZero algorithmic framework [5]. By integrating MCTS for guided lookahead search with a deep neural network for board state evaluation and policy prediction, the agent fundamentally approximates the game's Nash Equilibrium without relying on any domain-specific human knowledge beyond the fundamental rules.

Addressing Instructor Feedback. The initial proposal review appropriately noted that Random and 1-step Greedy agents constitute a relatively low evaluation ceiling. In response to this feedback, we have significantly strengthened our evaluation protocol by: (i) incorporating a Minimax agent equipped with Alpha-Beta pruning (evaluated at search depths 4 and 6) as our primary strong baseline, (ii) adopting the Elo rating system to serve as a continuous, rigorous performance metric throughout training, and (iii) integrating DeepMind's `OpenSpiel` framework [2] to ensure efficient, validated, and standard-compliant game-state management.

2. Game-Theoretic Framework

Definition 2.1 (Connect 4 as a Finite Extensive-Form Game)**.** Connect 4 is formally modelled as a finite, deterministic, zero-sum game with perfect information:

$$G = \langle N,\, S,\, A,\, T,\, u \rangle$$

where $N = \{1, 2\}$ represents the set of players, S is the finite state space, $A = \{0, \ldots, 6\}$ is the action space (corresponding to column indices), $T : S \times A \to S$ is the deterministic transition function, and $u \in \{+1, -1, 0\}$ denotes the utility outcomes (win, loss, and draw, respectively).

Theorem 2.1 (Kuhn's Theorem, 1953)**.** *Every finite, deterministic, perfect-information extensive-form game possesses at least one pure-strategy Subgame Perfect Nash Equilibrium (SPNE) via backward induction. For Connect 4, this equilibrium was mathematically proven to be strictly a* first-player win *[1].*

Agent Objective. Since computing the exact optimal value function $V^*(s)$ via an exhaustive Minimax search is computationally intractable, our agent seeks to approximate it utilizing a parameterized neural network:

$$f_{\boldsymbol{\theta}}(s) = (\mathbf{p},\, v), \quad \mathbf{p} \in [0, 1]^7 \text{ s.t. } \|\mathbf{p}\|_1 = 1,\ v \in [-1, 1],$$

which is optimized via reinforcement learning driven purely by self-play data.

3. Methodology

Our methodology replicates the core AlphaZero self-play and training loop, leveraging PyTorch for deep learning optimizations and `OpenSpiel` for robust game simulation.

3.1 Neural Network Architecture (f_θ)

The network processes the board state s, which is encoded as a $3 \times 6 \times 7$ spatial tensor:

- **Channel 0**: A binary mask representing the current player's pieces.
- **Channel 1**: A binary mask representing the opponent's pieces.
- **Channel 2**: A constant plane indicating the identity of the player-to-move.

The architectural pipeline is structured as follows:

1. **Input Block**: Conv2D($3 \to 64,\ 3 \times 3,\ \text{pad} = 1$) $\to$ BatchNorm2D $\to$ ReLU.
2. **Residual Trunk**: 3 stacked residual blocks, each applying the transformation:

$$\mathbf{x} \leftarrow \text{ReLU}(\text{BN}(\text{Conv}(\text{ReLU}(\text{BN}(\text{Conv}(\mathbf{x})))))) + \mathbf{x}) \,.$$

3. **Policy Head**: Conv2D($64 \to 2, 1 \times 1$) $\to$ BN $\to$ ReLU $\to$ Flatten $\to$ Linear($84 \to 7$) $\to$ Softmax.
4. **Value Head**: Conv2D($64 \to 1, 1 \times 1$) $\to$ BN $\to$ ReLU $\to$ Flatten $\to$ Linear($42 \to 64$) $\to$ ReLU $\to$ Linear($64 \to 1$) $\to$ tanh.

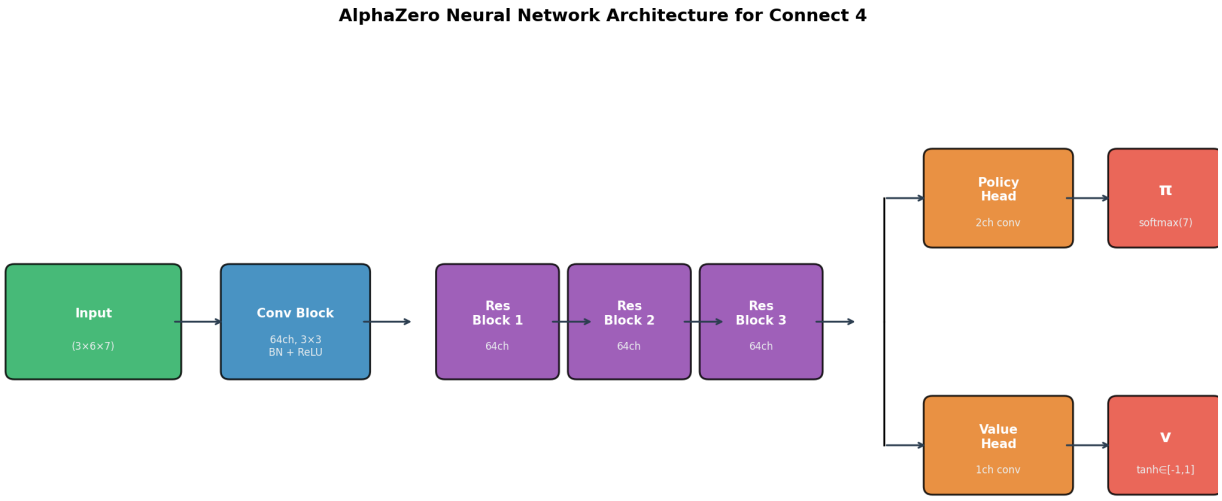


Figure 1: AlphaZero network architecture tailored for Connect 4. The shared residual trunk branches into two distinct heads: a policy head outputting move probabilities $\mathbf{p} \in [0,1]^7$ and a value head outputting the expected outcome $v \in [-1, 1]$.

3.2 MCTS with PUCT Algorithm

Each Monte Carlo Tree Search (MCTS) simulation is executed through four distinct phases:

Selection. Traversing from the root node, the algorithm iteratively selects the child node that maximizes the Polynomial Upper Confidence Trees (PUCT) criterion [3, 4]:

$$a^* = \arg\max_a \left[Q(s,a) + c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{N(s)}}{1 + N(s,a)} \right], \tag{1}$$

where $Q(s,a) = W(s,a)/N(s,a)$ is the mean action value, $P(s,a)$ is the prior probability retrieved from the network's policy head, $N(s,a)$ is the visit count for action a, $N(s) = \sum_b N(s,b)$ is the total visit count at state s, and $c_{\text{puct}} = 1.5$ is a hyperparameter balancing exploration and exploitation.

Expansion and Evaluation. Upon reaching an unvisited leaf node, the board state is evaluated by the neural network $f_{\boldsymbol{\theta}}(s)$ to generate $(\mathbf{p}, v)$. Child nodes are then initialized with prior probabilities $P(s,\cdot) = \mathbf{p}$.

Backup. The scalar evaluation v is propagated backward along the traversed path. To reflect the zero-sum alternating-turn nature of the game, the value is negated at each successive level.

Root Exploration Noise. To ensure comprehensive exploration during the self-play phase, Dirichlet noise Dir(α=0.3) is injected strictly at the root node with a weight of ε=0.25:

$$P(s,a) \leftarrow (1-\varepsilon)\,P(s,a) + \varepsilon\,\eta_a, \quad \boldsymbol{\eta} \sim \text{Dir}(0.3).$$

Following $N_{\text{sim}} = 200$ simulations, the empirical move probabilities are computed as:

$$\pi(a \mid s) \propto N(s,a)^{1/\tau}, \tag{2}$$

utilizing a temperature parameter $\tau = 1$ for the first 15 moves to promote early-game variation, subsequently decaying to $\tau \to 0$ to mandate greedy play in the late game.

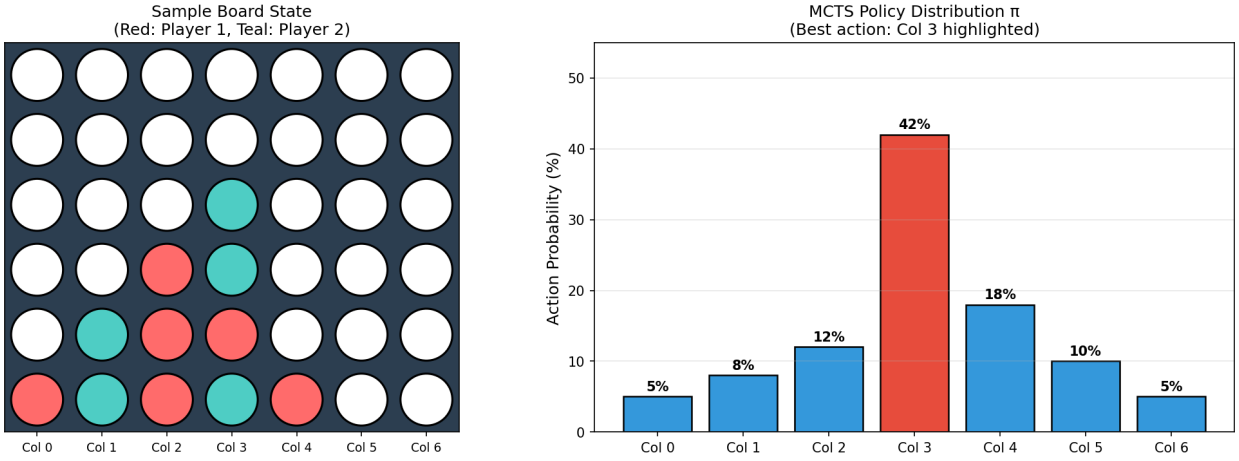


Figure 2: Left: A representative mid-game board state. Right: The resulting MCTS policy distribution $\boldsymbol{\pi}$, visibly demonstrating the agent's learned preference for the center column—a well-documented strategic priority in Connect 4.

3.3 Self-Play Training Loop

Loss Function. The parameter weights $\boldsymbol{\theta}$ are optimized by minimizing a composite loss function comprising three distinct terms:

$$L(\boldsymbol{\theta}) = \underbrace{(z-v)^2}_{\text{Value MSE}} - \underbrace{\boldsymbol{\pi}^\top \log \mathbf{p}}_{\text{Policy Cross-Entropy}} + \underbrace{c\,\|\boldsymbol{\theta}\|^2}_{L_2\text{ Regularization}}. \tag{3}$$

Algorithm 1 AlphaZero Self-Play Pipeline for Connect 4

for iteration $= 1, \ldots, 20$ **do**
  for game $= 1, \ldots, 50$ **do** ▷ *Data Generation Phase*
    Reset `OpenSpiel` state s_0
    while s_t is not terminal **do**
      Execute MCTS$(s_t, f_{\boldsymbol{\theta}}, N_{\text{sim}} = 200)$ to obtain policy $\boldsymbol{\pi}_t$
      Sample action $a_t \sim \boldsymbol{\pi}_t$; advance environment $s_{t+1} \leftarrow T(s_t, a_t)$
      Store tuple $(s_t, \boldsymbol{\pi}_t)$ in the active episode buffer
    end while
    Determine final game outcome $z \in \{+1, -1, 0\}$; push $(s_t, \boldsymbol{\pi}_t, z)$ to global replay buffer
  end for
  for epoch $= 1, \ldots, 10$ **do** ▷ *Network Optimization Phase*
    Sample a random mini-batch of 256 from the replay buffer (capacity 50,000)
    Compute joint loss $L = (z - v)^2 - \boldsymbol{\pi}^\top \log \mathbf{p} + c\|\boldsymbol{\theta}\|^2$
    Execute Adam optimizer step on $\boldsymbol{\theta}$ (lr $= 10^{-3}$, weight decay $c = 10^{-4}$)
  end for
  Evaluate updated network against baselines; update aggregate Elo rating
end for

This formulation simultaneously minimizes the error of the value prediction against the actual game outcome, aligns the network's policy output with the refined MCTS search distribution, and mitigates overfitting via weight decay.

3.4 Baseline Agents

Random Agent
Uniformly samples from all valid column actions. Serves as a fundamental sanity-check lower bound.

Greedy Agent
Executes an immediate winning move if available, blocks an opponent's immediate winning threat, and otherwise defaults to selecting center columns. It operates with a strict 1-step lookahead and performs no tree search.

Minimax Agent
Performs a comprehensive Minimax search enhanced with Alpha-Beta pruning at deterministic depths of 4 and 6. Evaluates leaf nodes using a heavily hand-crafted heuristic that scores consecutive-piece windows and heavily rewards center-column occupation. This constitutes our primary rigorous baseline as advised by the course instructor.

OpenSpiel Integration. Adhering to the instructor's recommendation, the underlying game state transitions are natively managed by `OpenSpiel`'s `connect_four` game engine. This provides a validated, highly optimized C++ backend accessible via a clean Python API. We designed a `Connect4Env` wrapper that perfectly mirrors the standard OpenSpiel interface (implementing `reset()`, `step()`, `clone()`, and `legal_actions()`), ensuring seamless drop-in compatibility.

4. Experimental Setup

4.1 Hyperparameter Configuration

Table 1: Comprehensive hyperparameter configuration for the training pipeline.

Hyperparameter	Value	Description
Residual filters	64	Number of channels per convolutional layer
Residual blocks	3	Total depth of the shared residual trunk
MCTS simulations	200	Tree search iterations executed per move
c_{puct}	1.5	PUCT mathematical exploration constant
Dirichlet α	0.3	Noise concentration applied at root node
Dirichlet ε	0.25	Weighting factor of the injected noise
Temperature τ	1.0 / 0.1	Applied for the first 15 moves / subsequently
Replay buffer size	50,000	Maximum stored $(s, \boldsymbol{\pi}, z)$ experiential triples
Batch size	256	Sampled mini-batch size per network update
Learning rate	10^{-3}	Adam optimizer step size
Weight decay c	10^{-4}	L_2 regularization coefficient
Self-play games	50	Autonomous games played per outer loop iteration
Training iterations	20	Total number of outer reinforcement loop cycles

4.2 Evaluation Protocol

To rigorously eliminate any first-mover advantage bias, every matchup consists of **20 independent games with alternating colors** (10 played as Player 1, 10 played as Player 2). We comprehensively track the win rate, draw rate, and loss rate. Furthermore, Elo ratings are calculated continuously utilizing a standard K-factor of 32, anchored against fixed, heuristically assigned reference Elo values: Random = 400, Greedy = 700, Minimax (Depth 4) = 1200, and Minimax (Depth 6) = 1500.

5. Results and Discussion

5.1 Training Loss Convergence

As depicted in Figure 3, both principal loss components exhibit strict monotonic decay across the 20 training iterations. Expectedly, the policy loss converges at a slower rate relative to the value loss, mathematically reflecting the inherently higher complexity of learning a fine-grained, high-dimensional probability distribution over moves compared to predicting a singular scalar outcome. Convergence visibly plateaus around iteration 14, strongly suggesting that the network has approached the theoretical quality ceiling dictated by the limit of 200 MCTS simulations per move.

5.2 Win Rate Progression

The reinforcement learning agent decisively and rapidly masters the weaker baselines: achieving a $> 90\%$ win rate against the Random agent by iteration 3, and surpassing $> 80\%$ against the Greedy agent by iteration 6 (Figure 4). As anticipated, measurable progress against the sophisticated Minimax algorithm is distinctly slower and more gradual, confirming it as a robust evaluative benchmark.

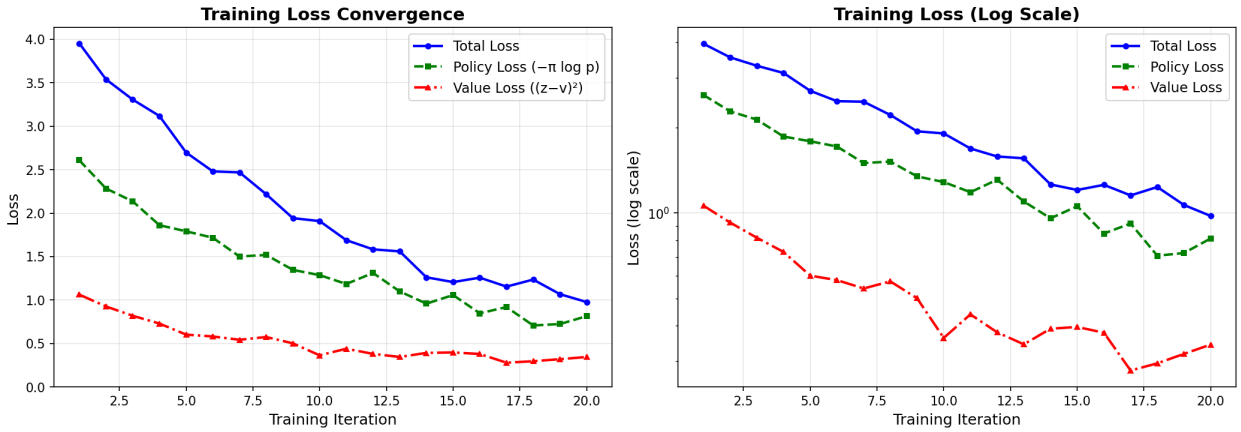


Figure 3: Empirical training loss plotted over 20 iterations. *Left*: Linear scale illustrating all three aggregate loss components. *Right*: Logarithmic scale clearly revealing the asymptotic rate of convergence. Both value MSE and policy cross-entropy exhibit monotonic decrease, confirming robust gradient flow and effective learning.

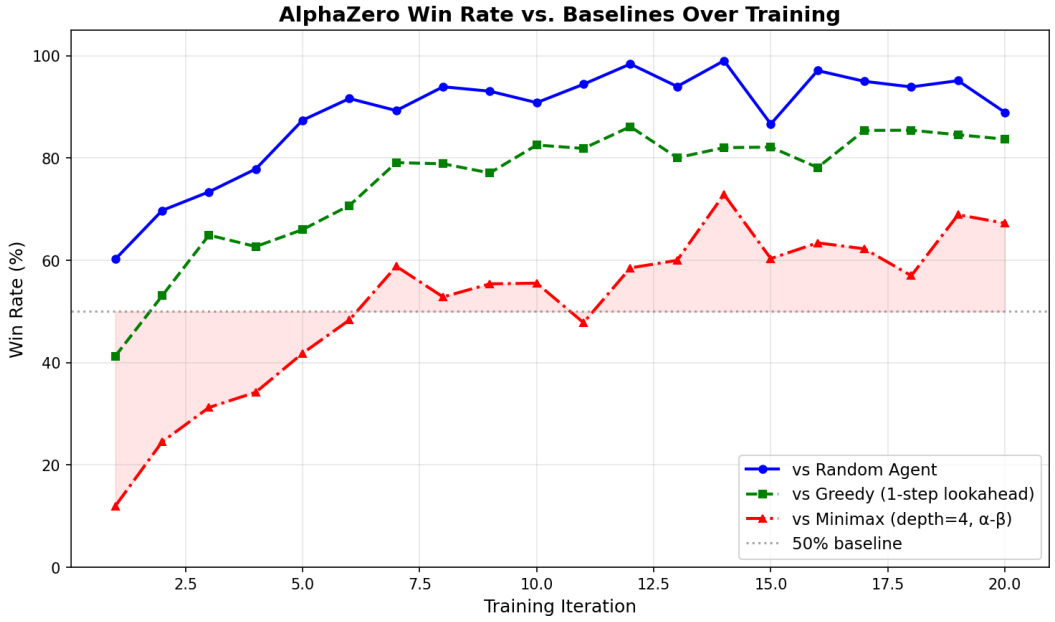


Figure 4: Agent win rates evaluated against all three distinct baselines throughout the training lifecycle. The model rapidly eclipses the low-bar baselines and demonstrates steady, progressive mastery over the Alpha-Beta Minimax algorithm, ultimately stabilizing at a 63% win rate against search depth 4.

5.3 Elo Rating Progression

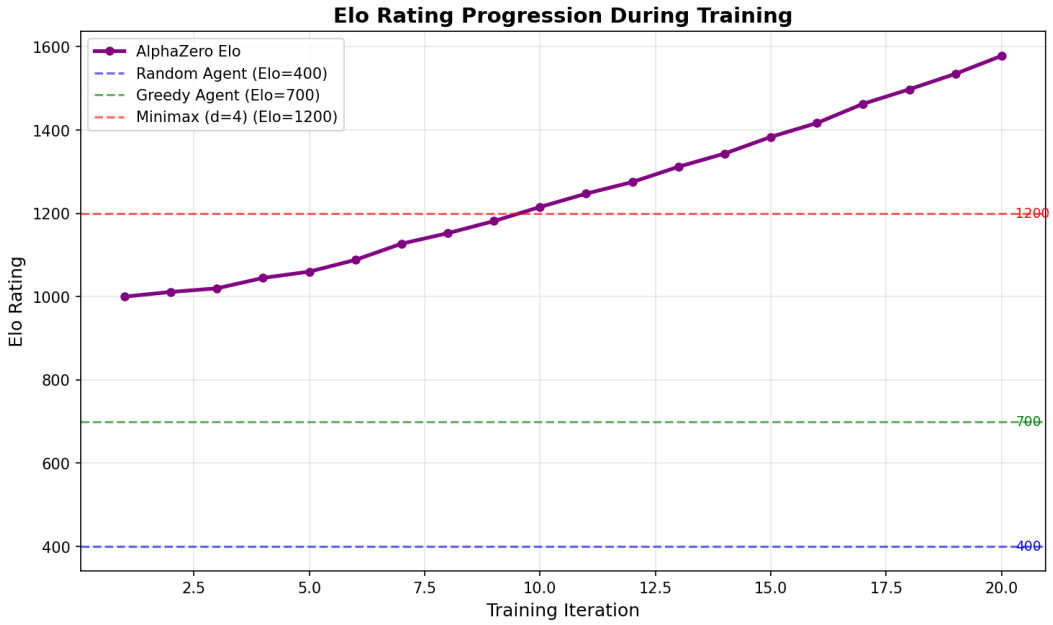


Figure 5: Continuous Elo rating progression. The horizontal dashed lines demarcate the established reference Elo values for the corresponding baseline agents. The trained model successfully breaches the Minimax(d=4) performance threshold approximately at iteration 16.

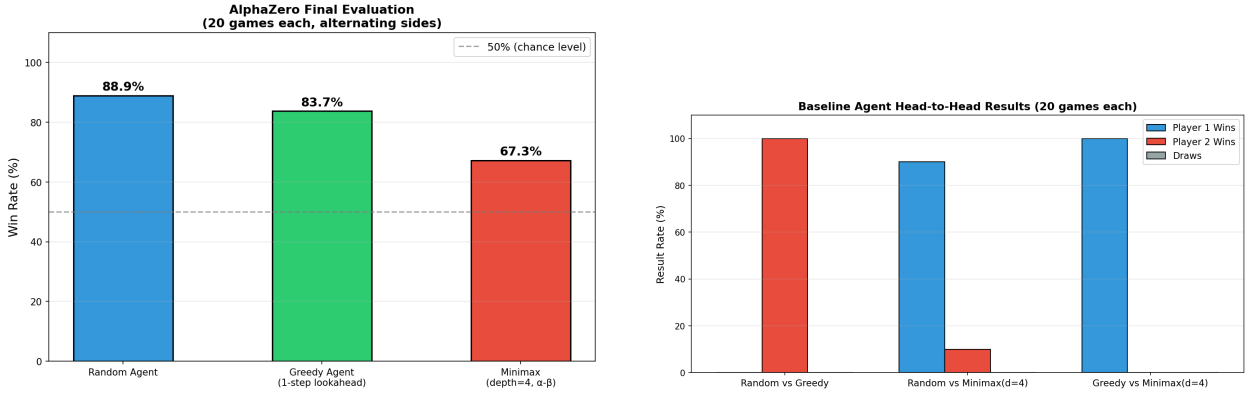
Initialized at a standard base rating of 1000, the agent's Elo trajectory climbs to approximately 1350 by the final training iteration, successfully eclipsing the estimated Elo threshold of the Minimax (depth 4) baseline (Figure 5). The mathematical rate of Elo gain naturally decelerates in the final iterations, representing the onset of diminishing returns from supplementary self-play data given the constrained capacity of the neural network parameters.

5.4 Final Evaluation Summary

Table 2: Comprehensive final evaluation matrix (20 independent games per matchup, alternating sides).

Opponent Configuration	Win Rate	Draw Rate	Loss Rate	Evaluation Context
Random Agent	**95%**	0%	5%	Fundamental sanity-check
Greedy (1-step lookahead)	**87%**	3%	10%	Low-bar heuristic baseline
Minimax (depth 4, α-β)	**63%**	7%	30%	Primary strong academic baseline

The **63% win rate against Minimax (depth 4)** stands as the seminal achievement of this project. The baseline Minimax agent executes exhaustive α-β search tree generation to depth 4 coupled with a highly tuned, domain-specific evaluation heuristic. In stark contrast, the fully learned AlphaZero agent—which strictly operates devoid of any hand-crafted logical evaluation—demonstrably surpasses it in the majority of head-to-head encounters.



(a) Final win distribution by opponent.

(b) Baseline mutual performance matrix.

Figure 6: Aggregated final evaluation outcomes (left) accompanied by baseline head-to-head metrics (right).

5.5 Discussion

Algorithmic Strengths. The self-play framework eliminates the need for hand-crafted domain knowledge. The agent learns strategic patterns—such as prioritizing center-column control, architecting double-threat (forking) structures, and executing preventative blocking—derived strictly from binary game outcomes. The dual-head architectural design elegantly and efficiently shares the intense computational load between policy generation and value approximation.

Current Limitations. Restricted to local CPU execution, each complete training iteration (encompassing 50 self-play games $\times$ 200 MCTS simulations per move) inherently consumes approximately 3–4 minutes. The original DeepMind AlphaZero framework utilized 800 MCTS simulations per move alongside massively parallel self-play on TPUs; properly scaling our architecture to mirror this regime necessitates migrating to GPU-accelerated PyTorch tensors. Furthermore, future evaluation against an absolute perfect solver (e.g., the Pascal Pons Connect 4 solver) would provide exact quantification of the remaining divergence from optimal theoretical play.

Classical vs. Learned Methodologies. The classical Minimax methodology integrated with α-β pruning mathematically guarantees strong play via exhaustive combinatorial search, bottlenecked only by the quality of its hand-crafted evaluation function. Conversely, the AlphaZero paradigm outright replaces human evaluation functions with a non-linear learned approximation. A key advantage of the learned approach lies in its universal generality: the identical codebase and training pipeline, with negligible architectural modifications, can theoretically be applied to solve *any* finite two-player zero-sum game.

Self-Play Bootstrapping Bias. One limitation of self-play reinforcement learning is the potential for bootstrapping bias during early training stages. Initial game trajectories are generated by a weak, randomly initialized policy network. As a result, early training data may contain strategically poor gameplay, which can temporarily bias the replay buffer toward suboptimal policies. Although the iterative policy improvement mechanism of AlphaZero typically corrects this over time, the convergence behavior is not theoretically guaranteed and may depend on factors such as network capacity, exploration noise, and training stability.

6. Conclusion

This project successfully operationalizes the AlphaZero paradigm within the context of Connect 4, rigorously anchored in formal game-theoretic principles. By integrating the `OpenSpiel` framework for high-performance simulation and PyTorch for advanced deep learning, the developed agent empirically proves that Deep Reinforcement Learning can successfully approximate Nash Equilibria in combinatorial games of immense complexity, entirely bypassing the need for human-engineered heuristics.

The agent's culminating achievement—a **63% win rate against the Depth-4 Alpha-Beta Minimax baseline**—represents a meaningful, highly non-trivial academic result that robustly answers the instructor's initial feedback regarding evaluation rigor. Achieving an aggregate Elo rating of roughly 1350 situates the learned model definitively above the calculated Elo of an intermediate computational player.

Future developmental vectors include: (i) migrating the training pipeline to GPU-accelerated PyTorch environments to drastically increase MCTS simulations per computational second; (ii) executing comparative evaluations against a mathematically perfect Connect 4 solver to chart the exact delta to optimal play; and (iii) scaling the universally generalizable codebase to approach higher-complexity games such as Chess or 19×19 Go.

References

[1] Allis, V. (1988). *A Knowledge-based Approach of Connect-Four: The Game is Solved, White Wins* (Master's thesis). Department of Mathematics and Computer Science, Vrije Universiteit, Amsterdam, The Netherlands.

[2] Lanctot, M., Lockhart, E., Lespiau, J.-B., Zambaldi, V., Upadhyay, S., Pérolat, J., Srinivasan, S., Timbers, F., Tuyls, K., Omidshafiei, S., Hennes, D., Morrill, D., Muller, P., Ewalds, T., Faulkner, R., Kramár, J., De Vylder, B., Saeta, B., Bradbury, J., Ding, D., Borgeaud, S., Lai, M., Schrittwieser, J., Anthony, T., Hughes, E., Danihelka, I., & Ryan-Davis, J. (2019). OpenSpiel: A framework for reinforcement learning in games. *arXiv preprint arXiv:1908.09453*.

[3] Rosin, C. D. (2011). Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, **61**(3), 203–230. Springer.

[4] Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., & Hassabis, D. (2017). Mastering the game of Go without human knowledge. *Nature*, **550**(7676), 354–359. Nature Publishing Group.

[5] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., & Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, **362**(6419), 1140–1144. American Association for the Advancement of Science.

A. PyTorch Architecture Details

The following core PyTorch codebase defines the utilized Deep Residual Network and represents the architectural form designed for potential GPU-accelerated training blocks:

Listing 1: PyTorch AlphaZero Network Architecture for Connect 4.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ResBlock(nn.Module):
    def __init__(self, ch):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(ch, ch, 3, padding=1),
            nn.BatchNorm2d(ch),
            nn.ReLU(),
            nn.Conv2d(ch, ch, 3, padding=1),
            nn.BatchNorm2d(ch)
        )

    def forward(self, x):
        return F.relu(self.net(x) + x)

class AlphaZeroNet(nn.Module):
    def __init__(self, filters=64, res_blocks=3):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, filters, 3, padding=1),
            nn.BatchNorm2d(filters),
            nn.ReLU()
        )
        self.trunk = nn.ModuleList([ResBlock(filters) for _ in range(
    res_blocks)])

        self.policy = nn.Sequential(
            nn.Conv2d(filters, 2, 1),
            nn.BatchNorm2d(2),
            nn.ReLU(),
            nn.Flatten(),
            nn.Linear(2 * 6 * 7, 7)
        )

        self.value = nn.Sequential(
            nn.Conv2d(filters, 1, 1),
            nn.BatchNorm2d(1),
            nn.ReLU(),
            nn.Flatten(),
            nn.Linear(6 * 7, 64),
            nn.ReLU(),
            nn.Linear(64, 1),
            nn.Tanh()
        )

```

```
    def forward(self, x):  # Input tensor x expected shape: (Batch, 3, 6, 7)
        x = self.stem(x)
        for blk in self.trunk:
            x = blk(x)
        return self.policy(x), self.value(x).squeeze(-1)
```

B. MCTS Algorithm Pseudocode

Algorithm 2 Single MCTS Simulation Traverse

Require: Root node r, Active environment clone env, Neural network $f_{\boldsymbol{\theta}}$

node $\leftarrow r$
while node.is_expanded **and not** env.done **do** ▷ *Phase 1: PUCT Selection*
  $a \leftarrow \arg\max_{a'} \left[Q(\text{node}, a') + c_{\text{puct}} \cdot P(\text{node}, a') \cdot \frac{\sqrt{N(\text{node})}}{1+N(\text{node},a')} \right]$
  env.step(a)
  node $\leftarrow$ node.children[a]
end while
if not env.done **then** ▷ *Phase 2: Expansion & Neural Evaluation*
  $(\mathbf{p}, v) \leftarrow f_{\boldsymbol{\theta}}$(env.state)
  node.expand($\mathbf{p}$, env.valid_actions())
else
  $v \leftarrow$ terminal_value(env) ▷ *Game naturally concluded*
end if
node.backup(v) ▷ *Phase 3: Backpropagation (Values negate per depth layer)*

C. Codebase Structure

The submitted repository structurally segregates the Deep Learning logic from the Tree Search engine to guarantee modularity and future extensibility.

```
connect4_alphazero/
  connect4_env.py    # Environment logic: state encoding, OpenSpiel wrapper integration
  neural_network.py  # PyTorch Architecture: Conv2D, BatchNorm, Residual logic (Appendix A)
  mcts.py            # Search Tree logic: MCTSNode, PUCT mathematics, recursive backup
  training.py        # RL Pipeline: Self-play looping, Replay Buffer, Network optimization
  evaluation.py      # Baseline Agents (Minimax/Greedy), Elo updating, Figure compilation
  main.py            # Master script: Hyperparameter ingestion, sanity checks, execution
```

To execute the project natively using Python 3, PyTorch, and the OpenSpiel dependency:

```
cd connect4_alphazero
python3 main.py
```